

詳細仕様書

1. システムの概要

1.1. このシステムの目的

このシステムは、家で運動したい人をサポートするアプリです。ユーザーは、いろいろな運動のやり方を見ながら、自分のポーズが正しいかリアルタイムで確認できます。また、運動の記録も残せます。健康的な生活を送る手助けをし、家での運動を習慣にするのが目的です。

1.2. 主な機能

このアプリには、次の機能があります。

- **ユーザー認証・管理:** ログイン、新しいアカウントの登録、パスワードの変更、自分の情報の編集が安全にできます。
- **運動情報の表示:** たくさんの運動の中から、それぞれの詳しい説明、写真を見ることができます。
- **リアルタイムポーズ検出・フィードバック:** カメラでユーザーのポーズをリアルタイムで見て、正しいポーズと比べて、画面や声でアドバイスします。
- **運動履歴の記録・表示:** 運動した時間、消費カロリーなどを自動で記録し、いつでも過去の運動履歴を確認できます。
- **参照ポーズを記録:** 自分のポーズをシステムに登録できます。

2. 画面について

2.1. 画面の一覧

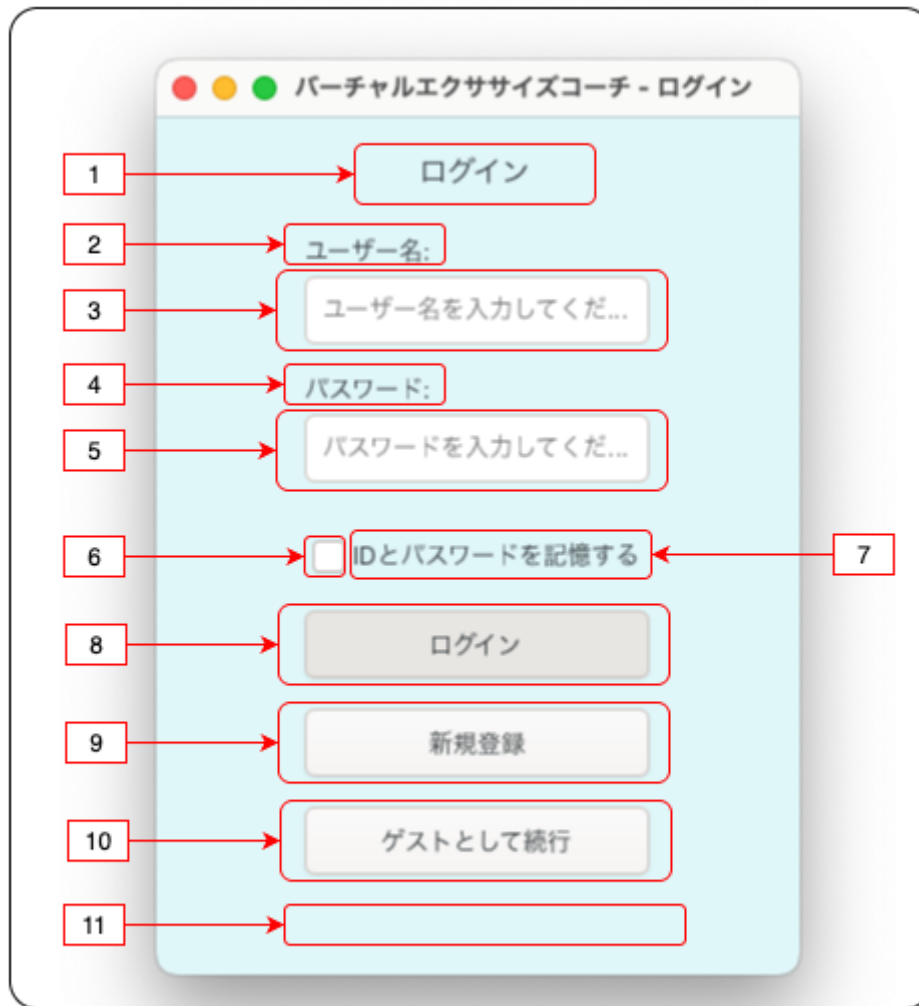
このシステムには、次の9つの画面があります。

- ログイン画面 (`LoginWindow`)
- 新規登録画面 (`RegisterWindow`)
- メイン画面（運動リスト） (`VirtualFitnessCoachWindow`)
- 運動詳細画面 (`ExerciseDetailWindow`)
- ポーズ検出画面 (`PoseDetectionWindow`)
- 運動履歴画面 (`ExerciseHistoryWindow`)
- マイページ画面 (`MyPageWindow`)
- ユーザー情報編集画面 (`ProfileEditWindow`)
- パスワード変更画面 (`PasswordChangeWindow`)

2.2. 各画面の説明

2.2.1. ログイン画面 (`LoginWindow`)

- **目的:** アプリを使うために、ユーザー名とパスワードで本人確認をします。
- **説明:** ユーザー名とパスワード入力フィールド、IDとパスワードを記憶するチェックボックス、ログインボタン、新規登録画面へ行くボタン、ゲストログインボタンがあります。



• UI部品:

- 1. `Gtk::Label` (固定文字列:"ログイン")
- 2. `Gtk::Label` (固定文字列:"ユーザー名:")
- 3. `Gtk::Entry` (ユーザー名入力フィールド)
- 4. `Gtk::Label` (固定文字列:"パスワード:")
- 5. `Gtk::Entry` (パスワード入力フィールド, `set_visibility(false)`で隠れます)
- 6. `Gtk::CheckButton` (IDとパスワードを記憶するチェックボックス)
- 7. `Gtk::Label` (固定文字列:"IDとパスワードを記憶する")
- 8. `Gtk::Button` (ログイン)
- 9. `Gtk::Button` (新規登録)
- 10. `Gtk::Button` (ゲストログイン)
- 11. `Gtk::Label` (エラーメッセージを表示)

• 操作の流れ:

1. ユーザーがユーザー名とパスワードを`Gtk::Entry`に入力します。
2. 「ログイン」ボタンを押します (`signal_clicked`)。
3. システムは入力された情報が正しいか、`AuthManager::authenticate_user`メソッドで確認します。
4. 認証が成功したら、`VirtualFitnessCoachWindow` (メイン画面) に移動します。
5. 認証が失敗したら、`Gtk::Label`にエラーメッセージ (例: 「ユーザー名かパスワードが違います」) を表示します。
6. 「新規登録」ボタンを押すと、`RegisterWindow`に移動します。

7. 「ゲストログイン」ボタンを押すと、ログインせずにVirtualFitnessCoachWindowに移動します。

2.2.2. 新規登録画面 (RegisterWindow)

- **目的:** 新しいユーザーアカウントを作ります。
- **説明:** 名、姓、ユーザー名、メールアドレス、パスワード、パスワード確認入力フィールド、登録ボタン、戻るボタンがあります。

The screenshot shows a window titled 'バーチャルエクササイズコーチ - 新規登録'. The form contains the following elements:

- 1. Title: 新規登録
- 2. Label: 名:
- 3. Input field: 名を入力してください
- 4. Label: 姓:
- 5. Input field: 姓を入力してください
- 6. Label: ユーザー名:
- 7. Input field: ユーザー名を入力してく...
- 8. Label: メールアドレス:
- 9. Input field: メールアドレスを入力し...
- 10. Label: パスワード:
- 11. Input field: パスワードを入力してく...
- 12. Label: パスワード確認:
- 13. Input field: パスワードを再入力して...
- 14. Button: 登録
- 15. Button: 戻る
- 16. Empty input field at the bottom.

- **UI部品:**
 - 1. `Gtk::Label` (固定文字列:"新規登録")
 - 2. `Gtk::Label` (固定文字列:"名:")
 - 3. `Gtk::Entry` (名入力フィールド)
 - 4. `Gtk::Label` (固定文字列:"姓:")
 - 5. `Gtk::Entry` (姓入力フィールド)
 - 6. `Gtk::Label` (固定文字列:"ユーザー名:")
 - 7. `Gtk::Entry` (ユーザー名入力フィールド)
 - 8. `Gtk::Label` (固定文字列:"メールアドレス:")
 - 9. `Gtk::Entry` (メールアドレス入力フィールド)

- 10. `Gtk::Label` (固定文字列:"パスワード:")
- 11. `Gtk::Entry` (パスワード入力フィールド, `set_visibility(false)`)
- 12. `Gtk::Label` (固定文字列:"パスワード確認:")
- 13. `Gtk::Entry` (パスワード確認入力フィールド, `set_visibility(false)`)
- 14. `Gtk::Button` (登録)
- 15. `Gtk::Button` (戻る)
- 16. `Gtk::Label` (エラーメッセージを表示)

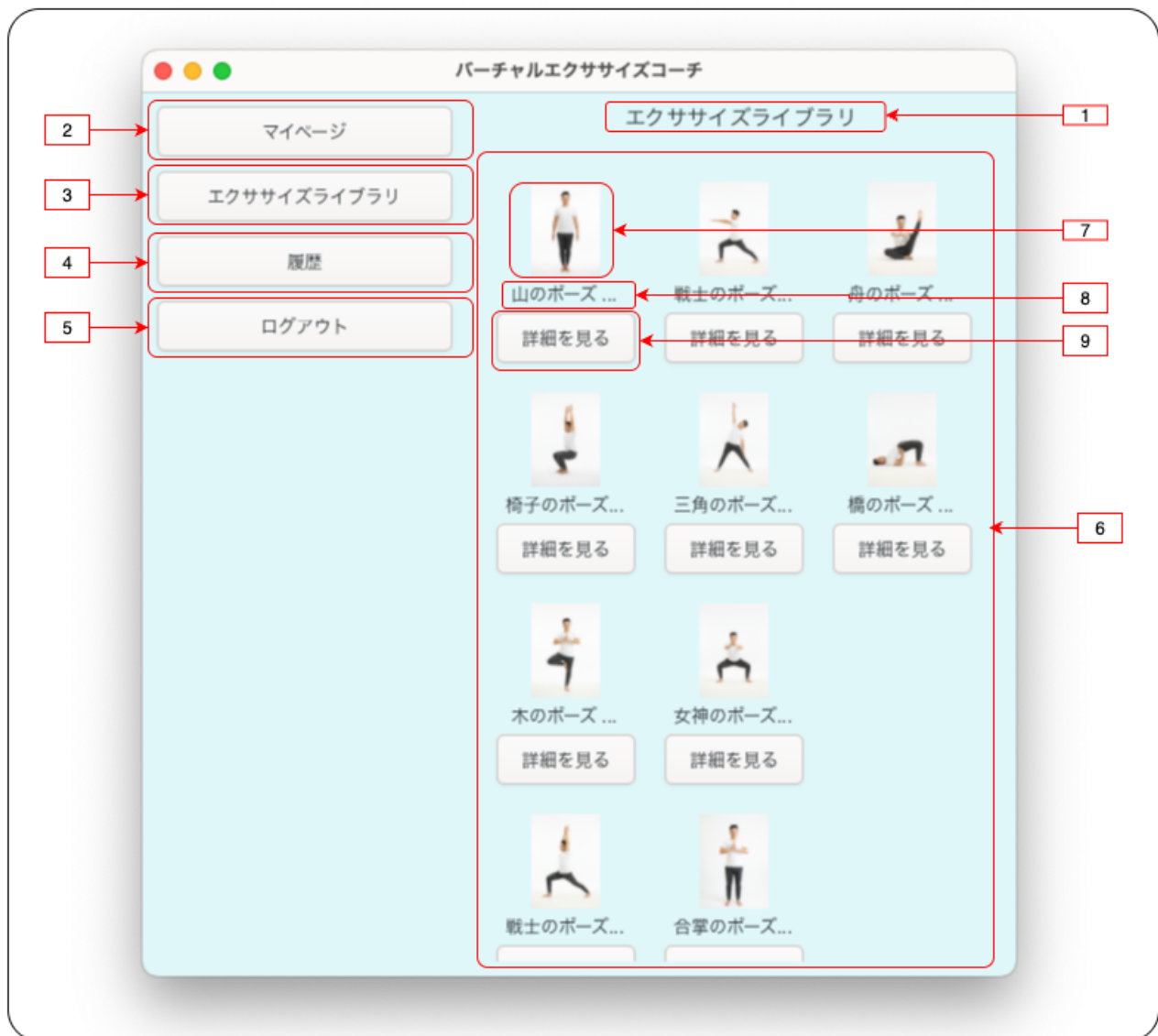
- 操作の流れ:

1. ユーザーがそれぞれの`Gtk::Entry`に情報を入力します。
2. 「登録」ボタンを押します (`signal_clicked`)。
3. システムは入力された情報が正しいか (例: パスワードが同じか、メールアドレスの形式が正しいか) 確認します。
4. `AuthManager::register_user`メソッドを呼び出し、ユーザー登録を試みます。
5. 登録が成功したら、`LoginWindow`に戻ります。
6. 登録が失敗したら、`Gtk::Label`にエラーメッセージ (例: 「ユーザー名かメールアドレスはすでに使われています」) を表示します。
7. 「戻る」ボタンを押すと、`LoginWindow`に戻ります。

2.2.3. メイン画面 (バーチャルフィットネスコーチ画面) (`VirtualFitnessCoachWindow`)

- 目的: 運動のリストを表示、ユーザーが運動を選べるようにします。
- 説明: 運動リストが表示され、それぞれの運動には名と写真があります。本画面にはマイページ、エクササイズライブラリ、履歴、ログアウトへ行くボタンがあります。

• UI部品:



- 1. `Gtk::Label` (固定文字列:"エクササイズライブラリ")
- 2. `Gtk::Button` (マイページ)
- 3. `Gtk::Button` (エクササイズライブラリ)
- 4. `Gtk::Button` (履歴)
- 5. `Gtk::Button` (ログアウト)
- 6. `Gtk::ListBox` (運動リストを表示する場所)
- 7. `Gtk::Image` (運動の写真、`exercises.image_path`)
- 8. `Gtk::Label` (運動の名、`exercises.name`)
- 9. `Gtk::Button` (詳細を見る)

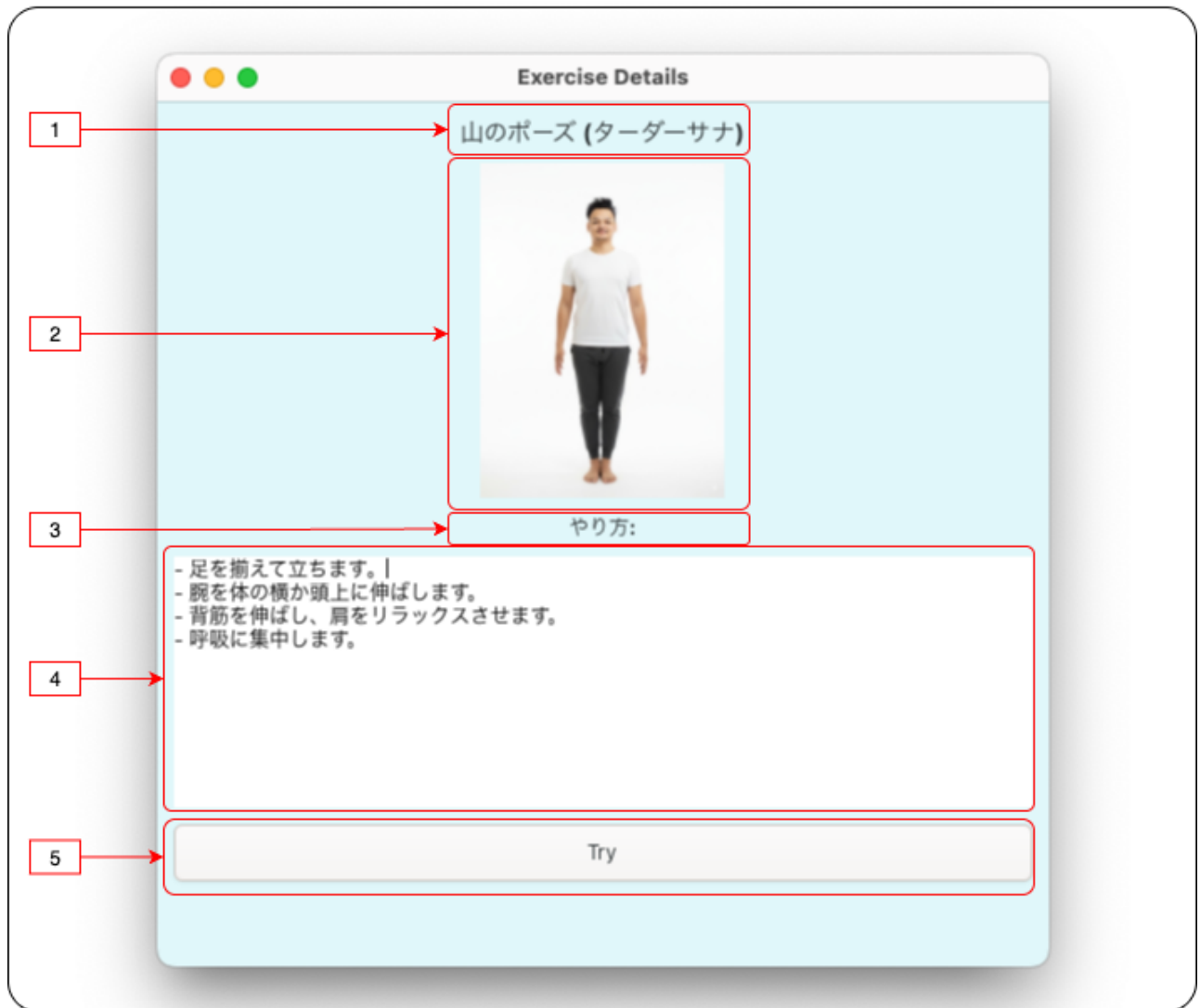
• 操作の流れ:

1. 画面が表示されたとき (`on_show`)、`DatabaseManager::fetch_exercises` メソッドを呼び出し、データベースから運動のデータを取得します。
2. 取得した運動データは `Gtk::ListBox` に表示されます。それぞれの行は `Exercise` オブジェクトに対応します。
3. ユーザーがリストの中から運動を選んで押すと (`signal_row_activated`)、選んだ運動のIDを `ExerciseDetailWindow` に渡して、詳細画面に移動します。
4. 「マイページ」ボタンを押すと、`MyPageWindow` に移動します。
5. 「履歴」ボタンを押すと、`ExerciseHistoryWindow` に移動します。

6. 「ログアウト」ボタンを押すと、`LoginWindow`に戻ります。

2.2.4. 運動詳細画面 (`ExerciseDetailWindow`)

- **目的:** 運動の詳しい情報を表示させて、ポーズ検出を始める入り口になります。
- **説明:** 運動の名、説明、やり方、写真などが表示されます。「ポーズ検出開始」ボタンがあります。
- **UI部品:**



- 1. `Gtk::Label` (運動の名、`exercises.name`)
 - 2. `Gtk::Image` (運動の写真、`exercises.image_path`)
 - 3. `Gtk::Label` (固定文字列:"やり方")
 - 4. `Gtk::ScrolledWindow` (運動のやり方)
 - 5. `Gtk::Button` (Try)
- **操作の流れ:**
 1. メイン画面から渡された運動IDを使用して、`DatabaseManager::fetch_exercises`を呼び出し、運動情報を取得します。
 2. 取得した情報をそれぞれの`Gtk::Label`や`Gtk::Image`に表示します。
 3. システムは`DatabaseManager::insert_exercise_session_start`を呼び出し、新しい運動セッションをデータベースに記録して、`exercise_history_id`を受け取ります。

- 「Try」ボタンを押すと、受け取った`exercise_history_id`と`Exercise`オブジェクトを`PoseDetectionWindow`に渡して、ポーズ検出画面に移動します。

2.2.5. ポーズ検出画面 (`PoseDetectionWindow`)

- **目的:** ユーザーのポーズをリアルタイムで表示して、アドバイスをしたり、参照ポーズを記録したりします。
- **説明:** カメラからの映像が`Gtk::DrawingArea`に表示され、その上に検出された体の骨格とアドバイスのメッセージが重なって表示されます。カウントダウン表示、記録ボタン、終了ボタンがあります。
- **UI部品:**

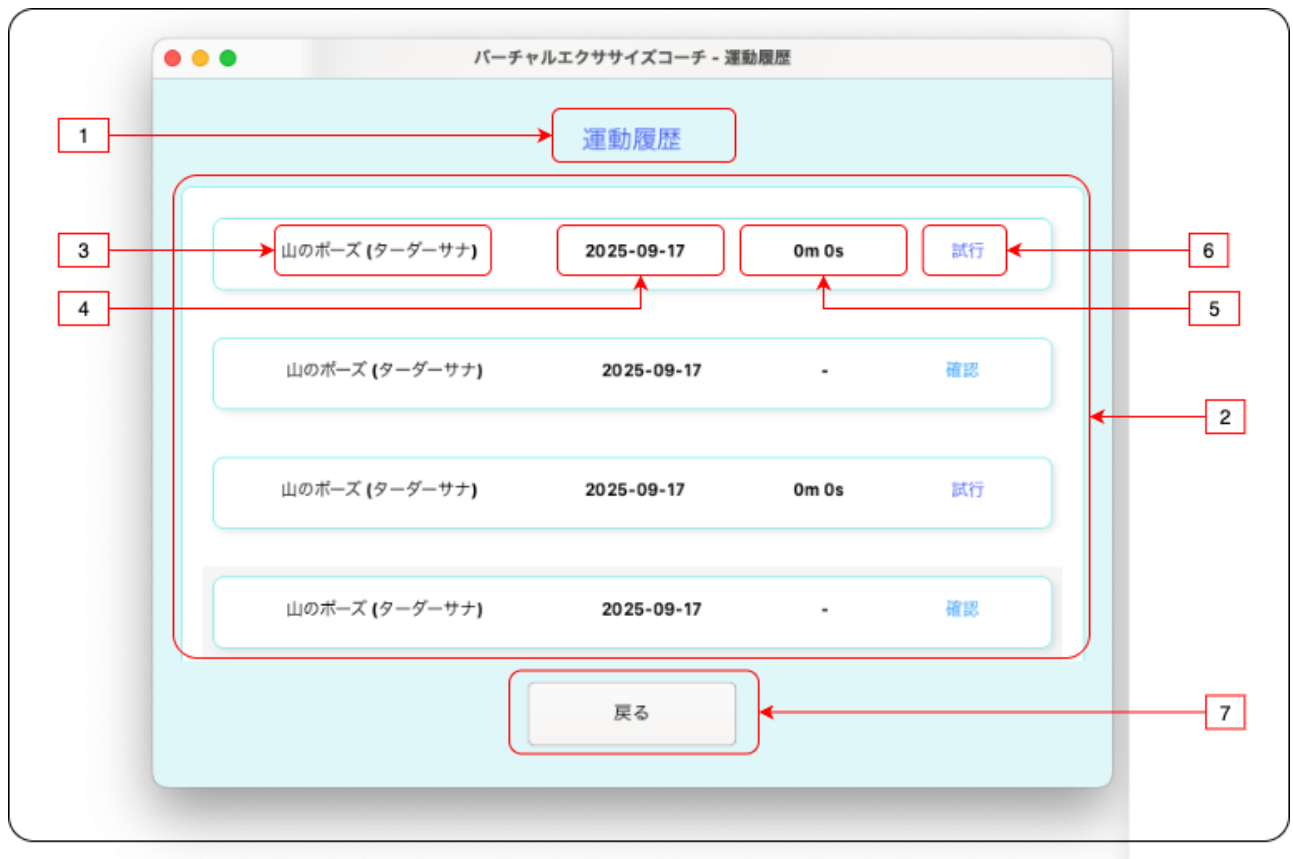


- 1. `Gtk::Label` (運動の名、`exercises.name`)
 - 2. `Gtk::Label` (案内メッセージ)
 - 3. `Gtk::DrawingArea` (カメラ映像とポーズ検出結果を描画する場所)
 - 4. `Gtk::Button` (参照ポーズを記録)
 - 5. `Gtk::Button` (閉じる)
- **操作の流れ:**
 1. 画面が表示されたとき (`on_realize`)、`start_camera`メソッドが呼ばれて、カメラ (`cv::VideoCapture`) が動きます。
 2. `Glib::signal_timeout().connect`で、約30FPS (33ms) ごとに`update_frame`メソッドが呼び出されます。
 3. `update_frame`の中で、カメラから映像のフレームを取得して、TensorFlow Liteモデル (`tf_lite_interpreter`) を使用してポーズを検出します。
 4. 検出された体のキーポイント (`last_detected_keypoints`) は、データベースから取得した参照ポーズ (`DatabaseManager::fetch_reference_pose`) と比べられます。

5. 比べた結果、エラーがあれば`error_message_queue`にエラーメッセージが追加され、`status_label`に表示されます。声でのアドバイス (`system("say -v Kyoko ...")`) もあります。
6. ポーズが正しい状態が続くと、`m_correct_pose_duration_ms`が増えていきます。
7. 「参照ポーズを記録」ボタンを押すと、5秒のカウントダウン (`on_countdown_timer`) の後、3秒間ポーズの記録 (`on_recording_timer`) が行われます。記録されたポーズの平均値は `DatabaseManager::insert_reference_pose` を使ってデータベースに保存されます。
8. 「閉じる」ボタンを押すと、カメラが止まり (`stop_camera`)、画面が閉じます。このとき、`m_correct_pose_duration_ms`を元に消費カロリーが計算され、`DatabaseManager::update_exercise_session_end`で運動履歴が更新されます。

2.2.6. 運動履歴画面 (`ExerciseHistoryWindow`)

- **目的:** ユーザー運動履歴をリストで表示します。
- **説明:** 運動名、運動した日、期間、状態 (例: 実行、試行) などが`Gtk::ListBox`に表示されます。
- **UI部品:**

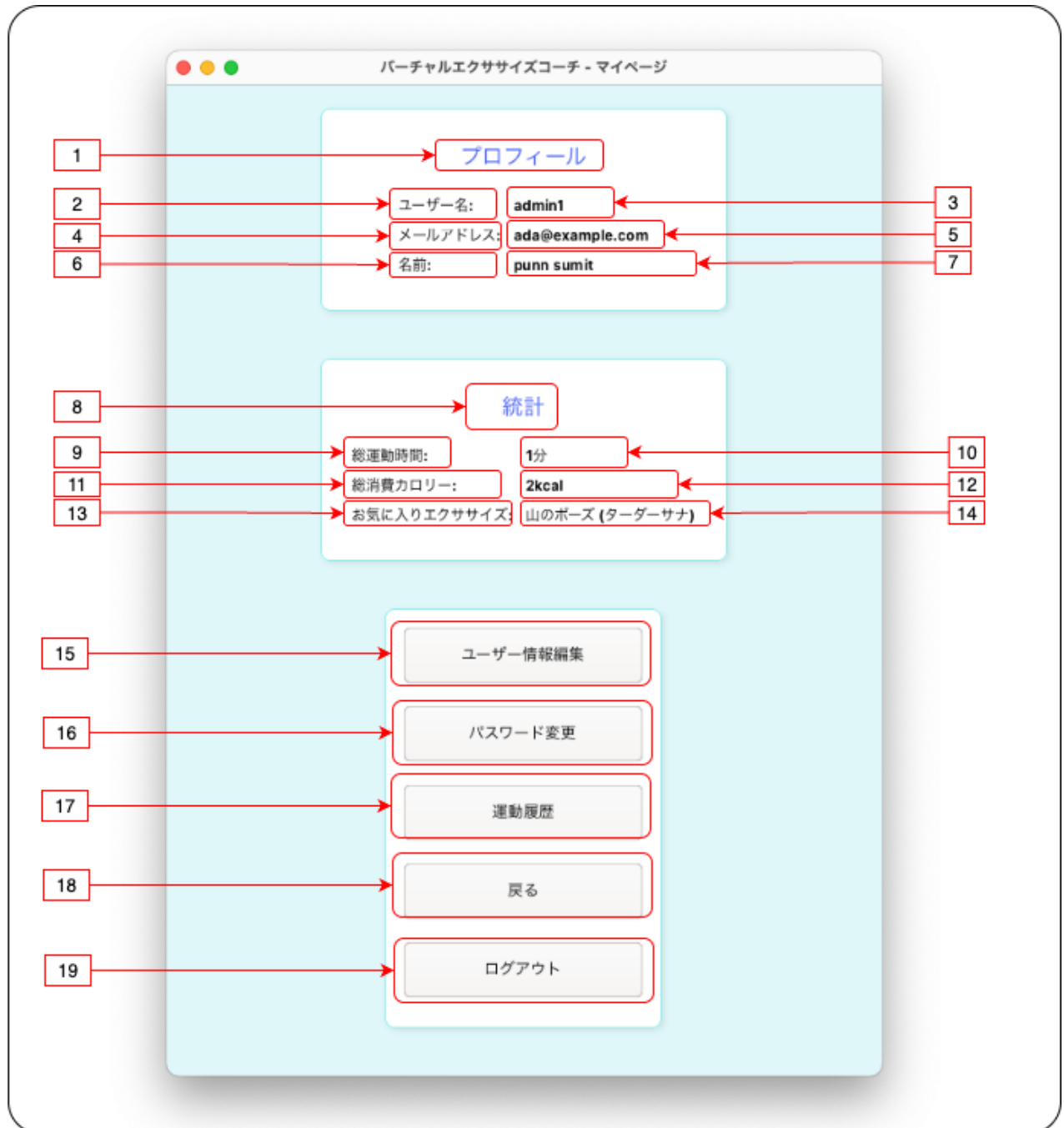


- 1. `Gtk::Label` (固定文字列:"運動履歴")
 - 2. `Gtk::ListBox` (運動履歴のリスト)
 - 3. `Gtk::Label` (運動名、`exercises.name`)
 - 4. `Gtk::Label` (運動した日、`exercise_history.session_date`)
 - 5. `Gtk::Label` (運動した期間、`exercise_history.duration_minutes`)
 - 6. `Gtk::Label` (状態、`exercise_history.status`)
 - 7. `Gtk::Button` (戻る)
- **操作の流れ:**
 1. 画面が表示されたとき (`on_show`)、`DatabaseManager::fetch_exercise_history`メソッドを呼び出し、今のユーザーの運動履歴を取得します。
 2. 取得した履歴データは`Gtk::ListBox`に表示されます。

3. 「戻る」ボタンを押すと、MyPageWindowがVirtualFitnessCoachWindowに戻ります。

2.2.7. マイページ画面 (MyPageWindow)

- **目的:** ユーザー情報や運動の合計データを表示し、関連する設定画面へ行けるようにします。
- **説明:** ユーザー名、メールアドレス、ユーザー氏名、合計運動時間、合計消費カロリーなどの情報が表示されます。ユーザー情報編集、パスワード変更、履歴表示、ログアウトへ行くボタンがあります。
- **UI部品:**

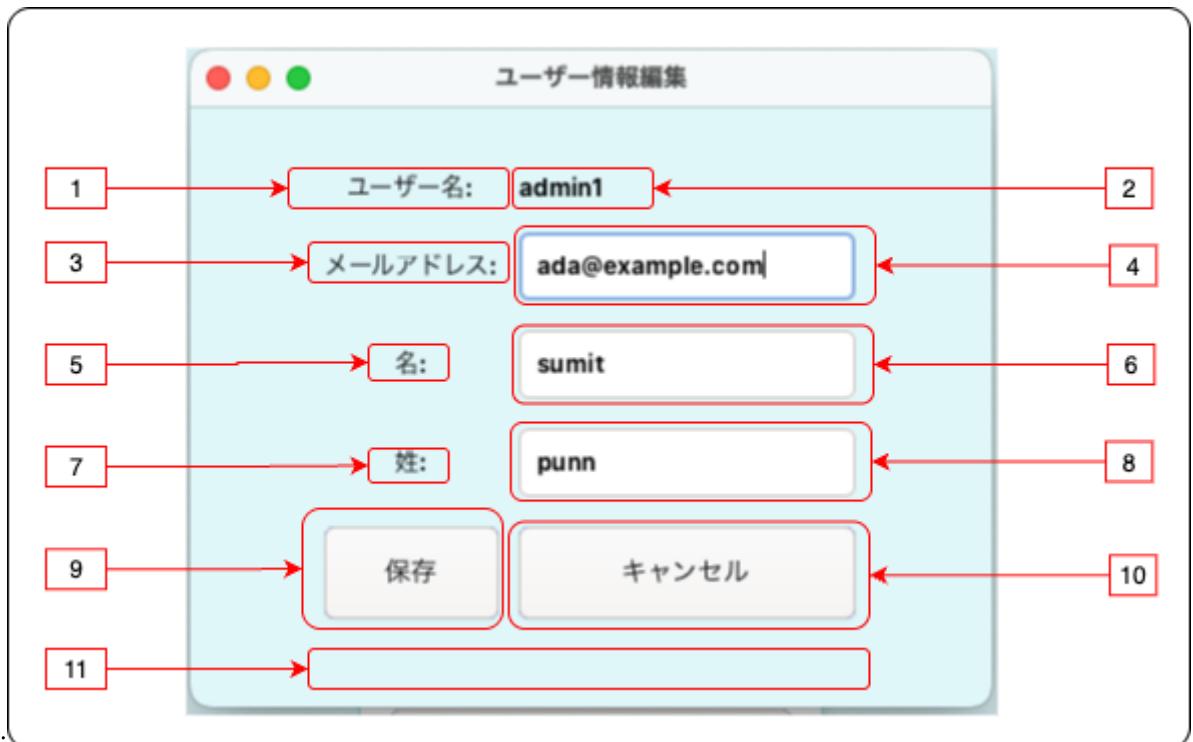


- 1. `Gtk::Label` (固定文字列:"プロフィール")
- 2. `Gtk::Label` (固定文字列:"ユーザー名:")
- 3. `Gtk::Label` (ユーザー名、users.username)
- 4. `Gtk::Label` (固定文字列:"メールアドレス:")
- 5. `Gtk::Label` (メールアドレス、users.email)
- 6. `Gtk::Label` (固定文字列:"名前:")
- 7. `Gtk::Label` (名前、users.first_name, users.last_name)

- 8. `Gtk::Label` (固定文字列:"統計")
 - 9. `Gtk::Label` (固定文字列:"総運動時間:")
 - 10. `Gtk::Label` (総運動時間、`exercise_history.duration_minutes`の合計)
 - 11. `Gtk::Label` (固定文字列:"総消費カロリー:")
 - 12. `Gtk::Label` (総消費カロリー、`exercise_history.calories_burned`の合計)
 - 13. `Gtk::Label` (固定文字列:"お気に入りエクササイズ:")
 - 14. `Gtk::Label` (お気に入りエクササイズ名、`user_profiles.preferences`から取得)
 - 15. `Gtk::Button` (ユーザー情報編集)
 - 16. `Gtk::Button` (パスワード変更)
 - 17. `Gtk::Button` (運動履歴)
 - 18. `Gtk::Button` (戻る)
 - 19. `Gtk::Button` (ログアウト)
- 操作の流れ:
 1. 画面が表示されたとき (`on_show`)、`DatabaseManager`からユーザー情報と運動の合計データ (例: `fetch_statistics`) を取得して、`Gtk::Label`に表示します。
 2. 「ユーザー情報編集」ボタンを押すと、`ProfileEditWindow`に移動します。
 3. 「パスワード変更」ボタンを押すと、`PasswordChangeWindow`に移動します。
 4. 「運動履歴」ボタンを押すと、`ExerciseHistoryWindow`に移動します。
 5. 「戻る」ボタンを押すと、`VirtualFitnessCoachWindow`に戻ります。
 6. 「ログアウト」ボタンを押すと、`LoginWindow`に戻ります。

2.2.8. ユーザー情報編集画面 (`ProfileEditWindow`)

- 目的: ユーザーのプロフィール情報を更新します。
- 説明: メールアドレス入力フィールド、名入力フィールド、姓入力フィールド、保存ボタン、キャンセルボタンがあります。



- UI部品:
 - 1. `Gtk::Label` (固定文字列:"ユーザー名:")
 - 2. `Gtk::Label` (ユーザー名、`users.username`)
 - 3. `Gtk::Label` (固定文字列:"メールアドレス:")

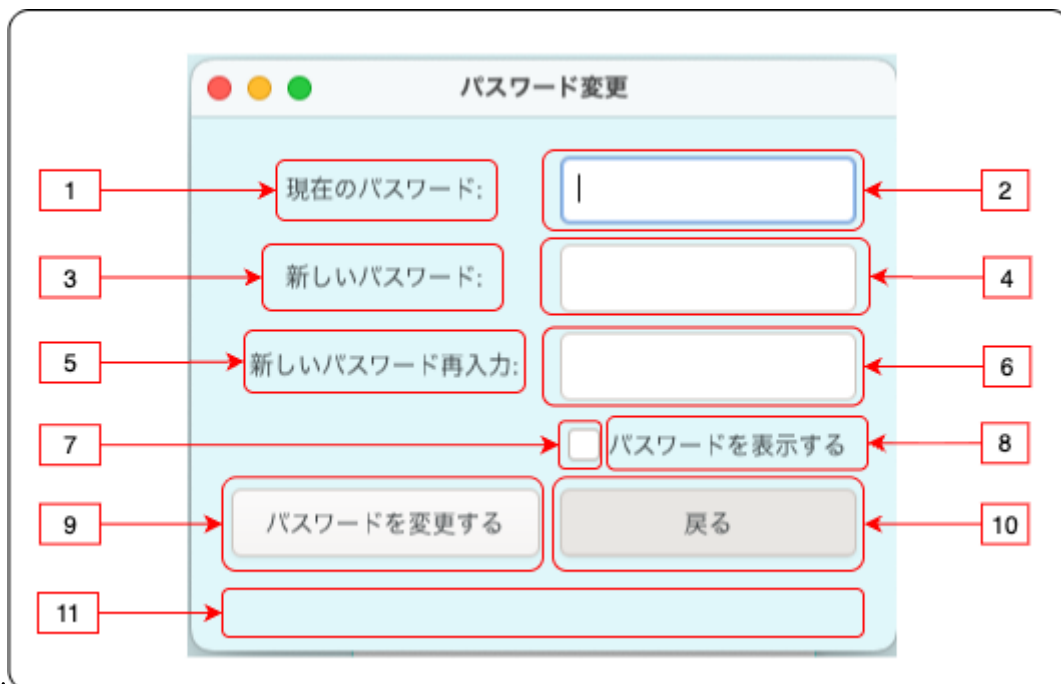
- 4. `Gtk::Entry` (メールアドレス入力フィールド、デフォルト値はusers.email)
- 5. `Gtk::Label` (固定文字列:"名:")
- 6. `Gtk::Entry` (名入力フィールド、デフォルト値はusers.first_name)
- 7. `Gtk::Label` (固定文字列:"姓:")
- 8. `Gtk::Entry` (姓入力フィールド、デフォルト値はusers.last_name)
- 9. `Gtk::Button` (保存)
- 10. `Gtk::Button` (キャンセル)
- 11. `Gtk::Label` (エラーメッセージを表示)

• 操作の流れ:

1. 画面が表示されたとき (`on_show`)、ユーザー情報 (`DatabaseManager::fetch_user_profile`) を取得して、それぞれのUI部品に表示します。
2. ユーザーが情報を編集して、「保存」ボタンを押します (`signal_clicked`)。
3. システムは入力された情報が正しいか確認し、`DatabaseManager::update_user_profile`メソッドを呼び出してデータベースを更新します。
4. 更新が成功したら、`MyPageWindow`に戻ります。
5. 更新が失敗したら、`Gtk::Label`にエラーメッセージを表示します。
6. 「キャンセル」ボタンを押すと、`MyPageWindow`に戻ります。

2.2.9. パスワード変更画面 (`PasswordChangeWindow`)

- 目的: ユーザーのパスワードを変更します。
- 説明: 現在のパスワード、新しいパスワード、新しいパスワード確認入力フィールド、変更ボタン、キャンセルボタンがあります。



• UI部品:

- 1. `Gtk::Label` (固定文字列:"現在のパスワード:")
- 2. `Gtk::Entry` (現在のパスワード入力フィールド, `set_visibility(false)`)
- 3. `Gtk::Label` (固定文字列:"新しいパスワード:")
- 4. `Gtk::Entry` (新しいパスワード入力フィールド, `set_visibility(false)`)
- 5. `Gtk::Label` (固定文字列:"新しいパスワード再入力:")
- 6. `Gtk::Entry` (新しいパスワード再入力フィールド, `set_visibility(false)`)
- 7. `Gtk::CheckButton` (パスワードを表示するチェックボックス)
- 8. `Gtk::Label` (固定文字列:"パスワードを表示する")

- 9. `Gtk::Button` (パスワードを変更する)
- 10. `Gtk::Button` (戻る)
- 11. `Gtk::Label` (エラーメッセージを表示する場所)
- 操作の流れ:
 1. ユーザーがそれぞれの`Gtk::Entry`にパスワードを入力します。
 2. 「パスワードを変更する」ボタンを押します (`signal_clicked`)。
 3. システムは入力された情報が正しいか (例: 新しいパスワードが同じか) 確認し、`AuthManager::change_password`メソッドを呼び出してパスワード変更を試みます。
 4. 変更が成功したら、`MyPageWindow`に戻ります。
 5. 変更が失敗したら、`Gtk::Label`にエラーメッセージを表示します。
 6. 「キャンセル」ボタンを押すと、`MyPageWindow`に戻ります。

3. 機能について

3.1. 機能の一覧

- ユーザー認証機能 (`AuthManager`)
- ユーザー情報管理機能 (`ProfileEditWindow`, `DatabaseManager`)
- 運動情報表示機能 (`VirtualFitnessCoachWindow`, `ExerciseDetailWindow`, `DatabaseManager`)
- ポーズ検出・アドバイス機能 (`PoseDetectionWindow`, TensorFlow Lite)
- 運動履歴記録・表示機能 (`ExerciseHistoryWindow`, `DatabaseManager`)
- 参照ポーズを記録機能 (`PoseDetectionWindow`, `DatabaseManager`)

3.2. 各機能の詳しい処理

3.2.1. ユーザー認証機能 (`AuthManager`)

- ログイン:
 - 入力: `username` (`Glib::ustring`), `password` (`Glib::ustring`)
 - 処理:
 1. `DatabaseManager::fetch_users`を呼び出し、入力された`username`と同じユーザーをデータベースから探します。
 2. ユーザーが見つかったら、`AuthManager::verify_password`メソッドを使って、入力された`password`とデータベースに保存されている`password_hash`を比べます。
 3. パスワードが同じなら、ユーザーIDを返します。
 4. 同じでない場合やユーザーが見つからない場合は、ログイン失敗とします。
 - 出力: ユーザーID (`int`) か、ログイン失敗を示す値
- 新規登録:
 - 入力: `username` (`Glib::ustring`), `email` (`Glib::ustring`), `password` (`Glib::ustring`)
 - 処理:
 1. 入力された`username`と`email`がデータベースにすでにないか、`DatabaseManager::fetch_users`で確認します。
 2. `AuthManager::hash_password`メソッドを使って、入力された`password`をハッシュ化します。
 3. `DatabaseManager::execute_query`を使って、新しいユーザー情報を`users`テーブルに入れます。
 - 出力: 登録成功/失敗 (`bool`)
- パスワード変更:

- 入力: `user_id` (int), `old_password` (Glib::ustring), `new_password` (Glib::ustring)
- 処理:
 1. `DatabaseManager::fetch_users`で`user_id`のユーザー情報を取ってきて、`AuthManager::verify_password`で`old_password`が正しいか確認します。
 2. `AuthManager::hash_password`で`new_password`をハッシュ化します。
 3. `DatabaseManager::execute_query`を使って、`users`テーブルの`password_hash`を更新します。
- 出力: 変更成功/失敗 (bool)

3.2.2. ユーザー情報管理機能 (`ProfileEditWindow`, `DatabaseManager`)

- ユーザー情報編集:
 - 入力: `User`オブジェクト (更新されユーザー情報が入っています)
 - 処理:
 1. `ProfileEditWindow`でユーザーが入力したユーザー情報を`User`オブジェクトに設定します。
 2. `DatabaseManager::update_user_profile`メソッドを呼び出し、`user_profiles`テーブルの対応する記録を更新します。
 - 出力: 更新成功/失敗 (bool)

3.2.3. 運動情報表示機能 (`VirtualFitnessCoachWindow`, `ExerciseDetailWindow`, `DatabaseManager`)

- 運動リスト表示:
 - 入力: なし
 - 処理:
 1. `VirtualFitnessCoachWindow`が始まったときや表示されたときに、`DatabaseManager::fetch_exercises`メソッドを呼び出し、`exercises`テーブルからすべての運動情報を`std::vector<Exercise>`として取得します。
 2. 取得した`Exercise`オブジェクトのリストを`Gtk::ListBox`に表示し、それぞれの運動の`name`と`image_path`を表示します。
 - 出力: `std::vector<Exercise>` (運動オブジェクトのリスト)
- 運動詳細表示:
 - 入力: `exercise_id` (int)
 - 処理:
 1. `ExerciseDetailWindow`は、渡された`exercise_id`を使って`DatabaseManager::fetch_exercises`を呼び出し、特定の運動情報を取得します。
 2. 取得した`Exercise`オブジェクトのそれぞれの情報 (`name`, `instructions`, `image_path` など) を画面の`Gtk::Label`や`Gtk::Image`に表示します。
 - 出力: `Exercise`オブジェクトの詳しい情報

3.2.4. ポーズ検出・アドバイス機能 (`PoseDetectionWindow`, TensorFlow Lite)

- 入力: カメラからのリアルタイム映像 (`cv::Mat`フレーム)
- 処理:
 1. `PoseDetectionWindow::update_frame`メソッドが約33msごとに動きます。

2. `cv::VideoCapture`から今のフレームを取ってきて、TensorFlow Liteモデル (`movenet_singlepose_lightning.tflite`) の入力形式に合わせて準備 (サイズ変更、正規化) をします。
 3. `tflite_interpreter->Invoke()`を呼び出し、ポーズ検出モデルを実行します。
 4. モデルが出した結果 (17個のキーポイントの座標と信頼度) を取ってきて、`last_detected_keypoints`に保存します。
 5. `DatabaseManager::fetch_reference_pose`を呼び出し、今運動している参照ポーズのデータを取得します。
 6. `last_detected_keypoints`と参照ポーズのそれぞれの関節の距離を計算し、`JOINT_ERROR_THRESHOLD`と比べます。
 7. 基準を超えた関節がある場合、`error_message_queue`にエラーメッセージ (例: 「左肩を直してください」) を追加し、`m_is_pose_correct`を`false`にします。
 8. `error_message_queue`が空の場合 (すべての関節が正しい場合)、`m_is_pose_correct`を`true`にし、`m_correct_pose_duration_ms`に33msを足します。
 9. `Gtk::DrawingArea`にカメラ映像、検出された骨格、そしてアドバイスメッセージを描画します。
 10. `m_is_pose_correct`が`REQUIRED_CORRECT_FRAMES`以上続いた場合、`DatabaseManager::update_exercise_session_end`を呼び出し、`performed_seconds` (`m_correct_pose_duration_ms`を秒に変換したもの) と計算された`calories_burned`で運動履歴を更新します。
- 出力: `cv::Mat` (骨格が描かれたフレーム), `Gtk::Label` (アドバイスメッセージ), 声でのアドバイス

3.2.5. 運動履歴記録・表示機能 (`ExerciseHistoryWindow`, `DatabaseManager`)

- 履歴記録:
 - 入力: `user_id` (int), `exercise_id` (int), `correct_performed_seconds` (int), `calories_burned` (int), `duration_minutes` (int), `status` (std::string)
 - 処理:
 1. 運動が始まったとき、`ExerciseDetailWindow`から `DatabaseManager::insert_exercise_session_start`を呼び出し、`exercise_history`テーブルに最初の記録を入れ、`m_exercise_history_id`を受け取ります。
 2. 運動が終わったとき (`PoseDetectionWindow::on_hide`) か、正しいポーズが一定時間続いた場合、`DatabaseManager::update_exercise_session_end`を呼び出し、`m_exercise_history_id`を使って`exercise_history`テーブルの記録を更新します。
 - 出力: なし
- 履歴表示:
 - 入力: `user_id` (int)
 - 処理:
 1. `ExerciseHistoryWindow`が表示されたとき (`on_show`)、`DatabaseManager::fetch_exercise_history`メソッドを呼び出し、`user_id`に関係するすべての運動履歴を`std::vector<std::map<std::string, std::string>>`として取得します。
 2. 取得した履歴データは`Gtk::ListBox`に表示されます。
 - 出力: `std::vector<std::map<std::string, std::string>>` (運動履歴のリスト)

3.2.6. 参照ポーズ記録機能 (`PoseDetectionWindow`, `DatabaseManager`)

- **入力:** カメラからのリアルタイム映像（ユーザーのポーズ）
- **処理:**
 1. `PoseDetectionWindow`で「参照ポーズを記録」ボタンが押されると、`record_reference_pose`メソッドが呼び出されます。
 2. すでに参照ポーズがあるか`DatabaseManager::fetch_statistics`で確認し、ユーザーに上書きでもいいか尋ねるダイアログ (`Gtk::MessageDialog`) を表示します。
 3. ユーザーが記録を続けると、5秒のカウントダウン (`on_countdown_timer`) が始まります。
 4. カウントダウンの後、`on_recording_timer`が3秒間、約100msごとに動き、`last_detected_keypoints`を`recorded_poses`に何回か保存します。
 5. 記録が終わった後、`recorded_poses`の中から真ん中のポーズを代表として選びます。
 6. `DatabaseManager::execute_query`で今ある参照ポーズを消し、`DatabaseManager::insert_reference_pose`を使って新しい参照ポーズのそれぞれの関節データ (`x, y, confidence`) を`reference_poses`テーブルに保存します。
- **出力:** 記録成功/失敗メッセージ (`Gtk::Label`)

4. データの仕様

4.1. 使うデータベース

- PostgreSQL (バージョン14を想定しています)

4.2. テーブルの仕様

4.2.1. `users` テーブル

- **説明:** アプリに登録されたユーザーの基本的な情報とログイン情報を管理します。
- **データ項目:**
 - `id`: SERIAL (主キー) - ユーザーを区別するための自動で振られる番号。
 - `username`: VARCHAR(50) (ユニーク, 必須) - ユーザーがログインに使う名前。50文字まで。
 - `email`: VARCHAR(100) (ユニーク, 必須) - ユーザーのメールアドレス。100文字まで。
 - `password_hash`: VARCHAR(255) (必須) - パスワードを暗号化したもの。セキュリティのため、元のパスワードは保存しません。
 - `first_name`: VARCHAR(50) - ユーザーの名前。50文字まで。
 - `last_name`: VARCHAR(50) - ユーザーの苗字。50文字まで。
 - `created_at`: TIMESTAMP (デフォルト: 現在時刻) - アカウントが作られた日時。
 - `updated_at`: TIMESTAMP (デフォルト: 現在時刻) - ユーザー情報が最後に更新された日時。
 - `last_login`: TIMESTAMP - ユーザーが最後にログインした日時。
 - `is_active`: BOOLEAN (デフォルト: TRUE) - アカウントが今使える状態かどうかのフラグ。

4.2.2. `user_sessions` テーブル

- **説明:** ユーザーのログイン状態を管理し、ログインを維持したりセキュリティを守ったりします。
- **データ項目:**
 - `id`: SERIAL (主キー) - セッションを区別するための自動で振られる番号。
 - `user_id`: INTEGER (外部キー: `users.id`, 必須) - このセッションがどのユーザーのものを示すID。 `users` テーブルの `id` とつながっています。
 - `token_hash`: VARCHAR(255) (必須) - 認証トークンを暗号化したもの。

- **expires_at**: TIMESTAMP (必須) - セッションがいつまで有効か。この時間を過ぎると使えなくなります。
- **created_at**: TIMESTAMP (デフォルト: 現在時刻) - セッションが作られた日時。
- **is_valid**: BOOLEAN (デフォルト: TRUE) - セッションが今使える状態かどうかのフラグ。

4.2.3. **user_profiles** テーブル

- **説明**: ユーザーの体の情報や運動に関する詳しいプロフィール情報を管理します。
- **データ項目**:
 - **id**: SERIAL (主キー) - プロフィールを区別するための自動で振られる番号。
 - **user_id**: INTEGER (外部キー: **users.id**, ユニーク, 必須) - このプロフィールがどのユーザーのものかを示すID。 **users** テーブルの **id** とつながっています。各ユーザーは1つのプロフィールだけ持てます。
 - **age**: INTEGER - ユーザーの年齢。
 - **height_cm**: INTEGER - ユーザーの身長（センチメートル）。
 - **weight_kg**: DECIMAL(5,2) - ユーザーの体重（キログラム）。小数点以下2桁まで。
 - **fitness_level**: VARCHAR(20) (デフォルト: 'beginner') - ユーザーの運動レベル（例: '初心者', '中級者', '上級者'）。
 - **goals**: TEXT - ユーザーの運動目標（自由に書けます）。
 - **preferences**: TEXT - ユーザーの運動に関する好み（自由に書けます）。
 - **created_at**: TIMESTAMP (デフォルト: 現在時刻) - プロフィールが作られた日時。
 - **updated_at**: TIMESTAMP (デフォルト: 現在時刻) - プロフィール情報が最後に更新された日時。

4.2.4. **exercises** テーブル

- **説明**: アプリで提供されるそれぞれの運動の詳しい情報を管理します。
- **データ項目**:
 - **id**: SERIAL (主キー) - 運動を区別するための自動で振られる番号。
 - **name**: VARCHAR(255) (必須) - 運動の名前。255文字まで。
 - **image_path**: VARCHAR(255) - 運動に関する写真ファイルの場所。255文字まで。
 - **category**: VARCHAR(100) - 運動の種類（例: 'ヨガ', '筋トレ'）。100文字まで。
 - **primary_muscle**: VARCHAR(100) - 主に鍛えられる筋肉。100文字まで。
 - **secondary_muscles**: TEXT - 補助的に鍛えられる筋肉（複数書けます）。
 - **equipment**: TEXT - 運動に必要な道具。
 - **difficulty_level**: VARCHAR(50) - 運動の難しさ（例: '簡単', '普通', '難しい'）。50文字まで。
 - **instructions**: TEXT - 運動の詳しいやり方。
 - **tips**: TEXT - 運動するときのヒントやコツ。
 - **video_url**: VARCHAR(255) - 運動の説明動画のURL。255文字まで。
 - **reps_sets_suggestion**: VARCHAR(255) - おすすめの回数やセット数。255文字まで。
 - **benefits**: TEXT - 運動で得られる効果。
 - **common_mistakes**: TEXT - 運動中によくある間違い。
 - **variations**: TEXT - 運動のバリエーション。
 - **calories_burned_estimate**: INTEGER - 1分間に消費されると予想されるカロリー（整数）。
 - **created_at**: TIMESTAMP (デフォルト: 現在時刻) - 運動情報が作られた日時。
 - **updated_at**: TIMESTAMP (デフォルト: 現在時刻) - 運動情報が最後に更新された日時。

4.2.5. **exercise_history** テーブル

- **説明:** 各ユーザーが運動した履歴を記録します。
- **データ項目:**
 - **id:** SERIAL (主キー) - 運動履歴を区別するための自動で振られる番号。
 - **user_id:** INTEGER (外部キー: **users.id**, 必須) - 運動したユーザーのID。
 - **exercise_id:** INTEGER (外部キー: **exercises.id**, 必須) - 行われた運動のID。
 - **session_date:** TIMESTAMP (デフォルト: 現在時刻) - 運動が始まった日時。
 - **duration_minutes:** INTEGER - 運動した合計時間 (分)。
 - **calories_burned:** INTEGER - この運動で消費されたと予想されるカロリー (整数)。
 - **notes:** TEXT - 運動に関するユーザーのメモ。
 - **status:** VARCHAR(50) (デフォルト: '確認') - 運動の達成状況 (例: '確認', '試行', '実行')。
 - **performed_seconds:** INTEGER - 正しいポーズを維持した合計時間 (秒)。この値でカロリーを計算します。

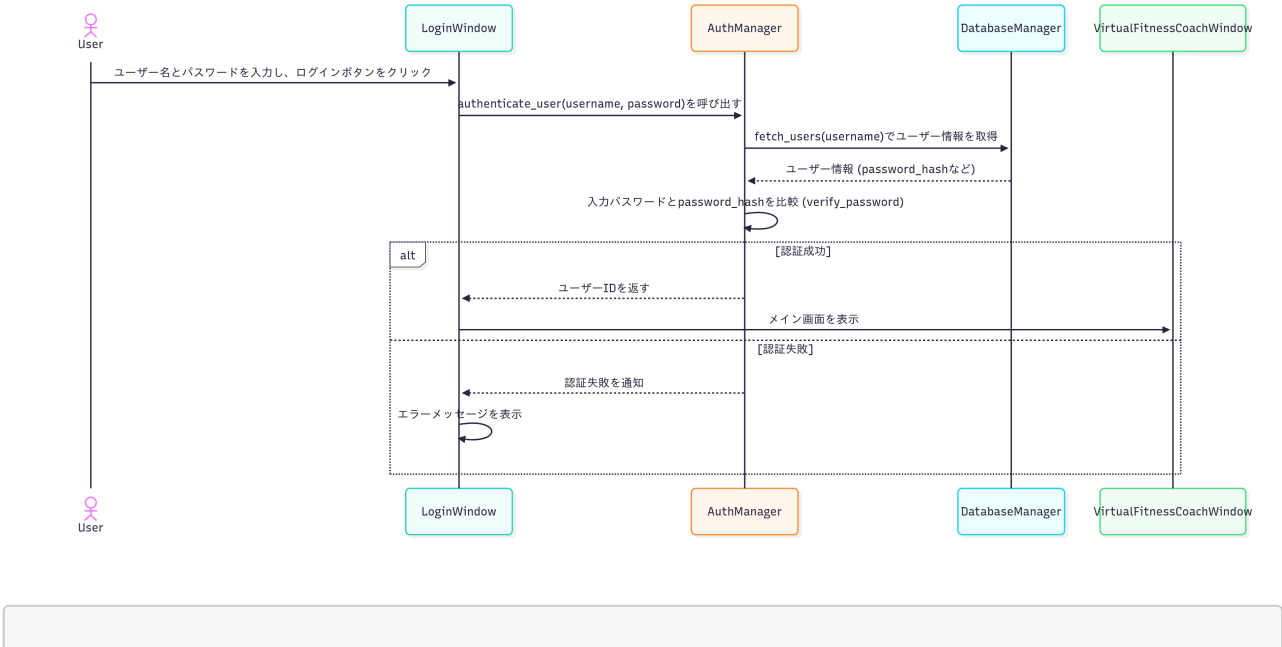
4.2.6. **reference_poses** テーブル

- **説明:** 各運動の「正しいポーズ」の基準となる関節の座標と信頼度を保存します。ポーズ検出で比べるために使われます。
- **データ項目:**
 - **id:** SERIAL (主キー) - 参照ポーズデータを区別するための自動で振られる番号。
 - **exercise_id:** INTEGER (外部キー: **exercises.id**, 必須) - この参照ポーズがどの運動のものを示すID。
 - **keypoint_index:** INTEGER (必須) - 検出された関節の番号 (例: 0:鼻, 1:左目, ..., 16:右足首)。
 - **x:** REAL (必須) - 関節のX座標。カメラ映像の幅に対する0.0~1.0の値。
 - **y:** REAL (必須) - 関節のY座標。カメラ映像の高さに対する0.0~1.0の値。
 - **confidence:** REAL (必須) - 検出された関節の信頼度 (0.0~1.0)。
 - **frame_number:** INTEGER (デフォルト: 0) - 参照ポーズが記録されたフレーム番号。静止ポーズの場合は通常0。

5. 処理の流れ

5.1. ログイン処理の流れ

- **説明:** ユーザーがアプリにログインするときの処理の流れです。

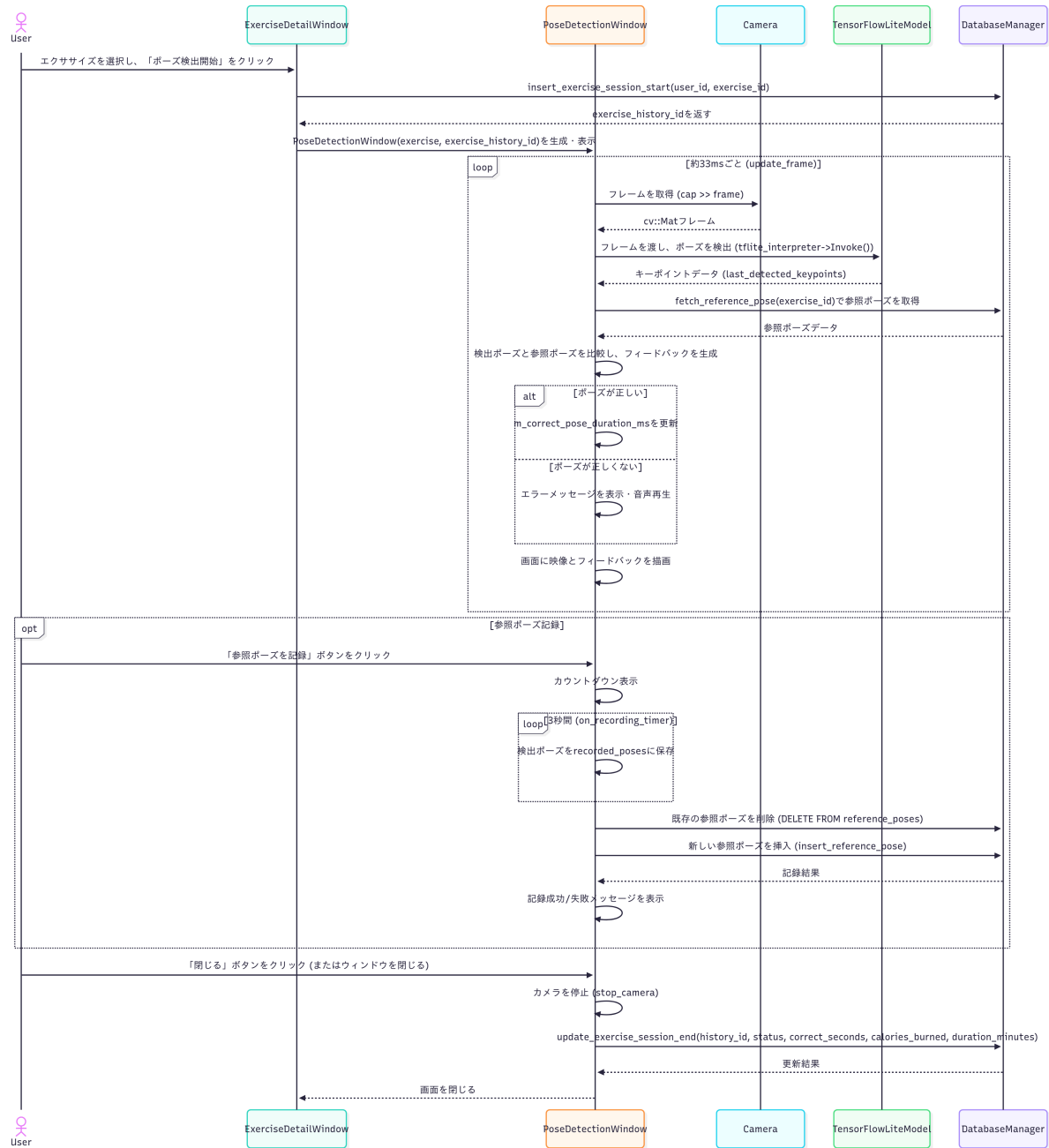


```
sequenceDiagram
    actor User
    participant LoginWindow
    participant AuthManager
    participant DatabaseManager
    participant VirtualFitnessCoachWindow

    User->>LoginWindow: ユーザー名とパスワードを入力し、ログインボタンをクリック
    LoginWindow->>AuthManager: authenticate_user(username, password)を呼び出す
    AuthManager->>DatabaseManager: fetch_users(username)でユーザー情報を取得
    DatabaseManager-->>AuthManager: ユーザー情報 (password_hashなど)
    AuthManager->>AuthManager: 入力パスワードとpassword_hashを比較 (verify_password)
    alt 認証成功
        AuthManager-->>LoginWindow: ユーザーIDを返す
        LoginWindow->>VirtualFitnessCoachWindow: メイン画面を表示
    else 認証失敗
        AuthManager-->>LoginWindow: 認証失敗を通知
        LoginWindow->>LoginWindow: エラーメッセージを表示
    end
end
```

5.2. ポーズ検出・記録処理の流れ

- 説明: ユーザーが運動を始めて、ポーズ検出と運動履歴の記録が行われるときの処理の流れです。



```
sequenceDiagram
    actor User
    participant ExerciseDetailWindow
    participant PoseDetectionWindow
    participant Camera
    participant TensorFlowLiteModel
    participant DatabaseManager
```

User->>ExerciseDetailWindow: エクササイズを選択し、「ポーズ検出開始」をクリック

ExerciseDetailWindow->>DatabaseManager:
insert_exercise_session_start(user_id, exercise_id)

DatabaseManager-->>ExerciseDetailWindow: exercise_history_idを返す

ExerciseDetailWindow->>PoseDetectionWindow:
PoseDetectionWindow(exercise, exercise_history_id)を生成・表示

```

loop 約33msごと (update_frame)
    PoseDetectionWindow->>Camera: フレームを取得 (cap >> frame)
    Camera-->>PoseDetectionWindow: cv::Matフレーム
    PoseDetectionWindow->>TensorFlowLiteModel: フレームを渡し、ポーズ
を検出 (tflite_interpreter->Invoke())
    TensorFlowLiteModel-->>PoseDetectionWindow: キーポイントデータ
(last_detected_keypoints)
    PoseDetectionWindow->>DatabaseManager:
fetch_reference_pose(exercise_id)で参照ポーズを取得
    DatabaseManager-->>PoseDetectionWindow: 参照ポーズデータ
    PoseDetectionWindow->>PoseDetectionWindow: 検出ポーズと参照ポーズ
を比較し、フィードバックを生成
    alt ポーズが正しい
        PoseDetectionWindow->>PoseDetectionWindow:
m_correct_pose_duration_msを更新
    else ポーズが正しくない
        PoseDetectionWindow->>PoseDetectionWindow: エラーメッセージを
表示・音声再生
    end
    PoseDetectionWindow->>PoseDetectionWindow: 画面に映像とフィードバ
ックを描画
end

opt 参照ポーズ記録
    User->>PoseDetectionWindow: 「参照ポーズを記録」ボタンをクリック
    PoseDetectionWindow->>PoseDetectionWindow: カウントダウン表示
loop 3秒間 (on_recording_timer)
    PoseDetectionWindow->>PoseDetectionWindow: 検出ポーズを
recorded_posesに保存
end
    PoseDetectionWindow->>DatabaseManager: 既存の参照ポーズを削除
(DELETE FROM reference_poses)
    PoseDetectionWindow->>DatabaseManager: 新しい参照ポーズを挿入
(insert_reference_pose)
    DatabaseManager-->>PoseDetectionWindow: 記録結果
    PoseDetectionWindow->>PoseDetectionWindow: 記録成功/失敗メッセー
ジを表示
end

    User->>PoseDetectionWindow: 「閉じる」ボタンをクリック (またはウィンドウ
を閉じる)
    PoseDetectionWindow->>PoseDetectionWindow: カメラを停止
(stop_camera)
    PoseDetectionWindow->>DatabaseManager:
update_exercise_session_end(history_id, status, correct_seconds,
calories_burned, duration_minutes)
    DatabaseManager-->>PoseDetectionWindow: 更新結果
    PoseDetectionWindow-->>ExerciseDetailWindow: 画面を閉じる

```

6. エラーや問題が起きたときの対応

6.1. 起こりうるエラー

- **データベース接続エラー:**
 - 起きる場所: `DatabaseManager`のコンストラクタや、データベースを操作するメソッド。
 - 原因: データベースサーバーが動いていない、接続情報が間違っている、インターネットの問題など。
- **ログイン失敗:**
 - 起きる場所: `AuthManager::authenticate_user`。
 - 原因: ユーザー名かパスワードがデータベースと一致しない。
- **ユーザー登録失敗:**
 - 起きる場所: `AuthManager::register_user`。
 - 原因: ユーザー名やメールアドレスがすでに使われている、入力した情報が正しくない。
- **ポーズ検出モデルの読み込み失敗:**
 - 起きる場所: `PoseDetectionWindow`のコンストラクタ。
 - 原因: `movenet_singlepose_lightning.tflite`ファイルが見つからない、またはファイルが壊れている。
- **カメラ起動失敗:**
 - 起きる場所: `PoseDetectionWindow::start_camera`。
 - 原因: カメラが見つからない、他のアプリがカメラを使っている、カメラのドライバーの問題。
- **参照ポーズが登録されていない:**
 - 起きる場所: `PoseDetectionWindow::update_frame`でポーズを比べるとき。
 - 原因: その運動の参照ポーズがデータベースに保存されていない。
- **データ入力エラー:**
 - 起きる場所: ユーザーが情報を入力する各画面（例: `RegisterWindow`, `ProfileEditWindow`, `PasswordChangeWindow`）。
 - 原因: ユーザーが必須項目を空にした、数字入力フィールドに文字を入れた、メールアドレスの形式が間違っているなど。

6.2. 対応の仕方

- **エラーメッセージの表示:**
 - 画面上で、エラーの内容を具体的に示すメッセージ（例: 「ユーザー名かパスワードが間違っています。」）を `Gtk::Label` に表示します。
 - `PoseDetectionWindow` では、`update_status_label` メソッドを使って、画面の下に赤い文字でエラーメッセージを表示します。
- **ログの出力:**
 - すべてのエラーは `std::cerr` に出力され、アプリのログファイル（もし設定されていれば）にも記録されます。これにより、開発者は問題の原因を詳しく調べることができます。
 - 例: `std::cerr << "Error: カメラを開けませんでした" << std::endl;`
- **再試行:**
 - データベース接続エラーなど、一時的な問題の可能性がある場合は、ユーザーにやり直すように促すメッセージを表示します。
- **初期値に戻す:**
 - ユーザー情報編集などで入力した情報が正しくない場合、できる範囲で前の値に戻すか、適切な初期値を設定します。
- **アプリの終了:**
 - ポーズ検出モデルの読み込み失敗やデータベースに接続できないなど、アプリを続けられないような重大なエラーが起きた場合は、ユーザーに知らせてから安全にアプリを終了します。

7. セキュリティ

7.1. ログインやアクセス権限の管理

- **パスワードのハッシュ化:**
 - ユーザーが登録したりパスワードを変えたりするとき、入力されたパスワードはそのままデータベースには保存されません。代わりに、`AuthManager::hash_password`メソッドの中で、SHA-256ハッシュアルゴリズムを使って暗号化されます。
 - ログインするときも、入力されたパスワードは同じように暗号化され、データベースに保存されている暗号化された値と`AuthManager::verify_password`メソッドで比べられます。これにより、パスワードが漏れるリスクを減らします。
- **セッション管理:**
 - ログインが成功すると、サーバー側でセッションIDが作られ、アプリにはJWT（JSON Web Token）のような認証トークンが発行されます。このトークンは`user_sessions`テーブルに暗号化されて保存され、有効期限が設定されます。
 - その後の通信ではこのトークンが使われ、サーバーはトークンが有効か確認することで、ユーザーのログイン状態やアクセス権限を管理します。

7.2. データの保護

- **データベースへのアクセス制限:**
 - アプリは、`DatabaseManager`クラスを通してのみPostgreSQLデータベースにアクセスします。直接データベースにアクセスすることは制限され、アプリが使うデータベースユーザーには、必要最低限の権限だけが与えられます。
- **入力情報の確認:**
 - ユーザーからの入力データは、データベースに保存される前に、各画面のUIの処理や`AuthManager`、`DatabaseManager`のメソッドの中で厳しくチェックされます。これにより、SQLインジェクションや不正なデータが入れられるなどのセキュリティリスクを減らします。
- **大事なデータの暗号化:**
 - パスワード以外の重要なデータ（例: 将来的に追加されるかもしれない個人を特定できる情報）については、データベースに保存するときや通信するときに暗号化することを考えます。